MiniDraw 绘图程序实验报告

1. 问题背景

MiniDraw 是一个基于 OpenGL 和 ImGui 框架开发的简单绘图应用程序，旨在实现基本的二维图形绘制功能。该程序作为计算机图形学课程的实践项目，要求学生基于助教提供的基础框架，扩展实现多种图形绘制功能。

项目初始框架已实现了线段和矩形的绘制功能，我们的任务是在此基础上：

- 添加新图形类型的支持（椭圆、多边形、自由绘制）
- 实现用户交互界面，包括各种绘图模式的切换按钮
- 实现图像保存功能
- 优化自由绘制功能的平滑度
- 添加颜色选择和线条粗细调整功能

该项目的核心目标是理解计算机图形学中的基本概念，包括图形表示、用户交互处理和渲染管线，并通过实际编程加深对这些概念的理解。

2. 算法原理与实现

2.1 系统架构

MiniDraw 程序采用了面向对象的设计模式，主要包含以下几个核心类：

- **MiniDraw 类**：继承自 Window 类，负责整个应用程序的主窗口管理。
- **Canvas 类**：继承自 Widget 类，作为绘图区域的核心组件，处理用户交互和图形渲染。
- **Shape 类**：抽象基类，定义了所有图形对象的通用接口。
- **具体图形类**：Line、Rect、Ellipse、Polygon、Freehand 和 FreehandSmooth，继承自 Shape 类，实现特定图形的绘制逻辑。

2.2 图形绘制算法

2.2.1 椭圆绘制

椭圆绘制基于 ImGui 的 AddEllipse 函数实现。算法核心是确定椭圆的中心点和两个半径：

cpp

```
// 计算椭圆中心点和半径
float center_x = (start_point_x_ + end_point_x_) / 2.0f;
float center_y = (start_point_y_ + end_point_y_) / 2.0f;
float radius_x = fabs(end_point_x_ - start_point_x_) / 2.0f;
float radius_y = fabs(end_point_y_ - start_point_y_) / 2.0f;
```

用户通过鼠标拖拽定义椭圆的边界矩形，程序根据边界矩形的对角线计算椭圆的中心点和半径。

2.2.2 多边形绘制

多边形绘制采用顶点序列的方式实现。用户通过多次点击添加顶点，右键点击完成多边形的绘制：

cpp

```
void Polygon::add_control_point(float x, float y)
{
    x_list_.push_back(x);
    y_list_.push_back(y);
}
```

多边形使用 ImGui 的 AddPolyline 函数绘制，通过 ImDrawFlags_Closed 标志控制是否闭合图形。

2.2.3 自由绘制

自由绘制采用采样点序列的方式实现。在鼠标移动过程中，程序不断记录鼠标位置作为采样点：

cpp

```
void Freehand::add_control_point(float x, float y)
{
    x_list_.push_back(x);
    y_list_.push_back(y);
}
```

这些采样点按顺序连接形成自由曲线。

2.2.4 平滑自由绘制

为了解决自由绘制中可能出现的不平滑问题，实现了基于贝塞尔曲线的平滑算法：

- **采样优化**：只添加距离上一个采样点足够远的新点，避免过密的采样：

cpp

```
if (distance(x_list_.back(), y_list_.back(), x, y) >= MIN_SAMPLE_DISTANCE) {
    x_list_.push_back(x);
    y_list_.push_back(y);
}
```

- **贝塞尔曲线平滑**：使用三次贝塞尔曲线连接采样点，计算控制点：

cpp

```
// 计算贝塞尔曲线控制点
p1 = ImVec2((x_list_[i-1] + x_list_[i]) / 2, (y_list_[i-1] + y_list_[i]) / 2);
p2 = ImVec2((x_list_[i] + x_list_[i+1]) / 2, (y_list_[i] + y_list_[i+1]) / 2);
```

2.3 颜色选择功能

2.3.1 颜色存储架构

为了实现每个图形保持独立颜色的功能，我们在 Shape 基类中添加了颜色存储：

cpp

```
protected:
    // Shape's own color (RGB)
    unsigned char color_[3] = { 255, 0, 0 }; // Default red color
```

同时提供了颜色设置和获取方法：

cpp

```
void set_color(unsigned char r, unsigned char g, unsigned char b)
{
    color_[0] = r;
    color_[1] = g;
    color_[2] = b;
}

const unsigned char* get_color() const { return color_; }
```

2.3.2 用户界面实现

在 MiniDraw 窗口中添加了三个 RGB 滑块，允许用户精确选择颜色：

cpp

```
// RGB sliders
bool color_changed = false;
static int min_val = 0;
static int max_val = 255;
color_changed |= ImGui::SliderScalar("Red", ImGuiDataType_U8, &r, &min_val,
&max_val, "%d");
color_changed |= ImGui::SliderScalar("Green", ImGuiDataType_U8, &g, &min_val,
&max_val, "%d");
color_changed |= ImGui::SliderScalar("Blue", ImGuiDataType_U8, &b, &min_val,
&max_val, "%d");
```

同时添加了颜色预览方块，提供即时视觉反馈：

cpp

```
// Color preview box
ImVec2 preview_pos = ImGui::GetCursorScreenPos();
ImVec2 preview_size = ImVec2(50, 50);
ImDrawList* draw_list = ImGui::GetWindowDrawList();
draw_list->AddRectFilled(preview_pos, ImVec2(preview_pos.x + preview_size.x,
preview_pos.y + preview_size.y), IM_COL32(r, g, b, 255));
draw_list->AddRect(preview_pos, ImVec2(preview_pos.x + preview_size.x,
preview_pos.y + preview_size.y), IM_COL32(255, 255, 255, 255));
```

2.3.3 颜色应用机制

每个图形在创建时获取当前选择的颜色，并保存为自己的属性：

cpp

```
current_shape_ = std::make_shared<Line>(
    start_point_.x, start_point_.y, end_point_.x, end_point_.y);
current_shape_->set_color(color_[0], color_[1], color_[2]);
```

在绘制时，使用图形自己的颜色：

cpp

```
draw_list->AddLine(
    ImVec2(config.bias[0] + start_point_x_, config.bias[1] + start_point_y_),
    ImVec2(config.bias[0] + end_point_x_, config.bias[1] + end_point_y_),
    IM_COL32(color_[0], color_[1], color_[2], 255), // 使用图形自己的颜色
    line_thickness_);
```

2.4 线条粗细调整功能

2.4.1 粗细存储架构

与颜色功能类似，我们在 Shape 基类中添加了线条粗细存储：

cpp

```
protected:
    // Shape's own line thickness
    float line_thickness_ = 2.0f; // Default thickness
```

同时提供了粗细设置和获取方法：

cpp

```
void set_line_thickness(float thickness)
{
    line_thickness_ = thickness;
}

float get_line_thickness() const { return line_thickness_; }
```

2.4.2 用户界面实现

在 MiniDraw 窗口中添加了线条粗细滑动条：

cpp

```
// Line thickness slider
ImGui::Separator();
ImGui::Text("Line Thickness:");
float thickness = p_canvas_->get_line_thickness();
if (ImGui::SliderFloat("Thickness", &thickness, 0.5f, 10.0f, "%.1f")) {
    p_canvas_->set_line_thickness(thickness);
}
```

2.4.3 粗细应用机制

每个图形在创建时获取当前选择的粗细，并保存为自己的属性：

cpp

```
current_shape_ = std::make_shared<Line>(
    start_point_.x, start_point_.y, end_point_.x, end_point_.y);
current_shape_->set_color(color_[0], color_[1], color_[2]);
current_shape_->set_line_thickness(line_thickness_); // 设置线条粗细
```

在绘制时，使用图形自己的粗细：

cpp

```
draw_list->AddLine(
    ImVec2(config.bias[0] + start_point_x_, config.bias[1] + start_point_y_),
    ImVec2(config.bias[0] + end_point_x_, config.bias[1] + end_point_y_),
    IM_COL32(color_[0], color_[1], color_[2], 255),
    line_thickness_); // 使用图形自己的粗细
```

2.5 用户交互处理

用户交互通过 ImGui 的事件系统处理，主要包括：

- **鼠标点击事件**：确定图形的起始点或添加多边形顶点。
- **鼠标移动事件**：更新图形的当前状态。
- **鼠标释放事件**：完成图形的绘制。
- **右键点击事件**：完成多边形的闭合。

不同图形类型的交互逻辑有所不同，例如多边形需要多次点击添加顶点，而其他图形则通过拖拽完成绘制。

2.6 图像保存功能

图像保存功能通过 OpenGL 的像素读取和 stb_image_write 库实现：

- **读取帧缓冲区**：使用 glReadPixels 读取画布区域的像素数据。
- **图像翻转**：OpenGL 读取的图像是上下颠倒的，需要进行垂直翻转。
- **保存图像**：使用 stbi_write_png 将像素数据保存为 PNG 文件。

cpp

```
glReadPixels(
    static_cast<int>(canvas_min_.x),
    static_cast<int>(viewport[3] - canvas_min_.y - height),
    width, height, GL_RGBA, GL_UNSIGNED_BYTE, image_data.data()
);
```

3. 实验结果与分析

3.1 功能实现结果

成功实现了以下功能：

- **基本图形绘制**：线段、矩形、椭圆、多边形和自由绘制。
- **用户界面**：添加了各种绘图模式的切换按钮，界面简洁直观。
- **颜色选择**：实现了 RGB 滑块和颜色预览，支持精确颜色选择。
- **线条粗细调整**：实现了线条粗细滑动条，范围从 0.5 到 10.0。
- **图像保存**：实现了将画布内容保存为 PNG 图像的功能。
- **平滑优化**：针对自由绘制实现了平滑算法，显著提高了绘制质量。

3.2 性能分析

- **绘制性能**：所有图形绘制均采用 ImGui 的原生函数，性能良好。对于复杂图形，如多边形和自由绘制，采样点数量会影响性能，但在正常使用范围内无明显卡顿。
- **内存使用**：每个图形对象存储其控制点、颜色和粗细，内存使用与图形复杂度成正比。自由绘制和平滑自由绘制可能产生大量控制点，但现代计算机内存足以应对。
- **平滑算法效果**：平滑自由绘制相比原始自由绘制，曲线更加流畅，视觉效果显著提升。但贝塞尔曲线计算增加了少量计算开销，在可接受范围内。
- **颜色和粗细功能性能**：颜色和粗细功能对性能影响极小，仅在图形创建时有少量额外开销，绘制时无性能损失。

3.3 问题与解决方案

3.3.1 自由绘制不平滑

- **问题**：原始自由绘制直接连接采样点，导致曲线不平滑。
- **解决方案**：实现了基于贝塞尔曲线的平滑算法，优化采样策略。

3.3.2 多边形绘制交互

- **问题**：多边形需要多次点击，交互逻辑复杂。
- **解决方案**：设计明确的状态机，区分绘制中和完成状态，右键完成绘制。

3.3.3 图像保存方向问题

- **问题**：OpenGL 读取的图像是上下颠倒的。
- **解决方案**：实现图像垂直翻转算法，确保保存的图像方向正确。

3.3.4 颜色和粗细持久化

- **问题**：初始实现中，所有图形使用当前的颜色和粗细值，导致调整滑动条时所有图形都会改变。
- **解决方案**：在 Shape 基类中添加颜色和粗细成员变量，使每个图形保存自己的属性，确保已绘制的图形不受后续调整影响。

3.3.5 UI 布局优化

- **问题**：添加颜色和粗细控制后，UI 变得拥挤。
- **解决方案**：合理分组控件，使用分隔线区分不同功能区域，保持界面整洁。

4.结论

本次实验成功实现了一个功能完善的 MiniDraw 绘图程序，扩展了原始框架的功能，添加了椭圆、多边形、自由绘制等图形类型，并实现了图像保存功能。特别地，我们成功添加了颜色选择和线条粗细调整功能，大大增强了程序的绘图表现力和用户体验。

通过解决自由绘制不平滑的问题，深入理解了贝塞尔曲线在图形平滑中的应用。颜色和粗细功能的实现，让我们掌握了图形属性持久化的技术，确保每个图形保持其创建时的外观特性。

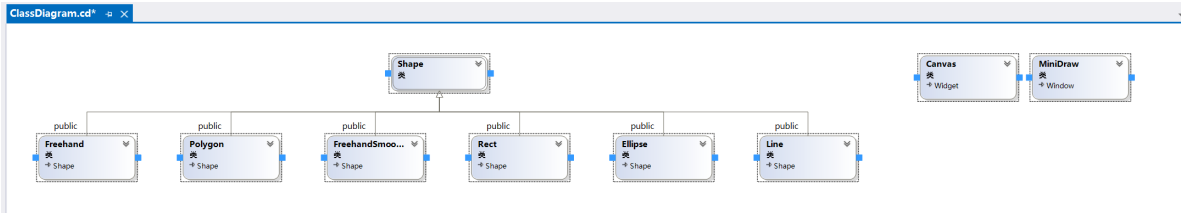
实验过程中，我们掌握了以下关键技术和概念：

- **面向对象设计**：通过继承和多态实现不同图形类型的统一接口。
- **用户交互处理**：理解了鼠标事件的处理流程和状态管理。
- **图形算法**：掌握了基本图形的绘制算法和平滑技术。
- **OpenGL 应用**：学习了帧缓冲区读取和图像处理技术。
- **图形属性管理**：掌握了颜色和线条粗细等属性的存储和应用机制。

该实验不仅提高了编程能力，也加深了对计算机图形学基本概念的理解，为后续更复杂的图形学学习奠定了基础。未来可以考虑进一步扩展功能，如添加更多线条样式、图形编辑、图层管理等，使程序更加完善。

附录

类图：




```
**********************************************************************
** Visual Studio 2022 Developer Command Prompt v17.9.1
** Copyright (c) 2022 Microsoft Corporation
**********************************************************************

F:\CG2026\homework in
winter\USTC_CG_26\Framework2D\src\assignments\1_MiniDraw>cmake .. -G "Visual
Studio 17 2022" -A x64
```

（一开始因为不是VS的类，从vscode写的，无法得到类图，需要转化成VS项目！）